Energy-Efficient Distributed Nanopore Basecalling for Mobile Edge Health Systems

Michał Ławinski
Department of Applied Informatics
Silesian University of Technology
Gliwice, Poland

Dariusz Mrozek
dariusz.mrozek@polsl.pl
Department of Applied Informatics
Silesian University of Technology
Gliwice, Poland

Abstract

Nanopore sequencing enables portable genomic analysis in field and mobile healthcare settings (e.g., rural areas), but real-time basecalling remains computationally demanding on low-power edge platforms. This paper investigates performance–energy trade-offs of distributed genomic basecalling on a Kubernetes-orchestrated cluster of NVIDIA Jetson Nano devices with node- and batch-level parallelism. Three basecalling models—SACall, RODAN, and Chiron—were evaluated across different batch sizes and power modes for throughput, energy consumption, resource utilization, and scalability. Experimental results show that SACall provides the best overall balance between throughput, scalability, and power efficiency, while RODAN achieves the highest throughput for small workloads and Chiron demonstrates near-linear scaling at the cost of high GPU utilization. The results confirm the feasibility of energy-efficient genomic inference on mobile edge infrastructures and highlight the importance of jointly optimizing AI workloads, power usage, and distributed execution in resource-constrained health applications.

CCS Concepts

• **Computer systems organization** → **Distributed architectures**; • **Human-centered computing** → *Empirical studies in ubiquitous and mobile computing*; • **Applied computing** → *Computational genomics.*

Keywords

Edge computing, mobile genomics, nanopore sequencing, basecalling, resource consumption, throughput analysis, healthcare

ACM Reference Format:
Michał Ławinski and Dariusz Mrozek. 2026. Energy-Efficient Distributed Nanopore Basecalling for Mobile Edge Health Systems. In *The 32nd Annual International Conference on Mobile Computing and Networking (MobiCom '26), October 26–30, 2026, Austin, TX, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3704413.3765514

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MobiCom '26, Austin, Texas, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1353-8/25/10
https://doi.org/10.1145/3704413.3765514

1 Introduction

Nanopore sequencing is a widely used DNA and RNA sequencing technology that enables rapid, real-time genomic analysis in both laboratory and field environments [2]. Basecalling, i.e., the transformation of raw electrical signals into nucleotide sequences, has evolved from statistical approaches such as Nanocall [8] toward deep learning-based architectures including Chiron [22], RODAN [14], and SACall [11]. Although these models significantly improve sequencing accuracy, they also impose substantial computational and energy demands, particularly in portable, resource-constrained deployments in healthcare and life sciences.

Portable nanopore sequencing has already demonstrated applicability in field laboratories, epidemic surveillance, and remote environments with limited infrastructure availability [3, 5]. Such mobile health scenarios often require local processing due to unstable connectivity, latency constraints, and limited access to cloud resources. Consequently, edge computing has emerged as a promising approach for performing genomic inference closer to the data source while reducing dependence on centralized infrastructure [12, 13]. At the same time, the high computational cost of deep neural networks (DNNs) and their impact on hardware efficiency remain important challenges for low-power edge platforms [21]. Previous studies investigated basecalling on individual edge devices, including NVIDIA Jetson platforms and dedicated accelerators, primarily focusing on throughput and energy efficiency [9, 20, 24]. While these works confirm the feasibility of edge-based genomics, they are typically limited to single-node deployments and do not address distributed execution, orchestration, or horizontal scalability across edge clusters.

Containerization and orchestration frameworks such as Kubernetes [4] provide mechanisms for workload distribution, automated scaling, and fault-tolerant execution, making them attractive for mobile edge bioinformatics deployments. Despite growing interest in edge AI systems, the literature still lacks systematic studies jointly evaluating basecalling models, power modes, and batch-level parallelism in distributed edge environments. In particular, the combined impact of these factors on throughput, energy consumption, and CPU/GPU utilization remains insufficiently explored.

This paper addresses these gaps by evaluating distributed nanopore basecalling on a Kubernetes-orchestrated cluster of low-power edge devices. The study analyzes how batch size, power modes, and parallel execution jointly affect performance, scalability, and energy efficiency across multiple deep learning (DL) basecalling models. The presented results provide practical insights into the design of energy-efficient mobile edge infrastructures for real-time genomic analysis and portable health applications.

2 Related Work

Many nanopore basecallers, including Nanocall [8], relied on statistical approaches and CPU-based inference, but were limited in accuracy and scalability. Also, existing studies on Guppy, Bonito, and Dorado [15, 16, 19], widely adopted in practical sequencing workflows, do not focus on distributed execution and energy efficiency. Deep learning significantly improved basecalling performance, with architectures such as Chiron [22], RODAN [14], and SACall [11] leveraging recurrent and attention-based models for more accurate signal-to-sequence translation. Although their architectures are well-studied in other AI domains, their scalability and energy characteristics in genomic basecalling on edge platforms remain insufficiently analyzed.

Energy-efficient AI has been widely investigated through model compression, hardware-aware optimization, and low-power inference techniques [6, 7, 21]. Edge computing enables AI inference closer to data sources, reducing latency and cloud dependency [12, 13]. Recent studies demonstrated genomic analysis on platforms such as Jetson Xavier NX [9]; however, most works focus on isolated edge devices and general-purpose workloads rather than distributed genomic inference. In particular, the interaction between model architecture, batch size, and energy consumption remains poorly understood. Also, container orchestration platforms such as Kubernetes [4] are increasingly used for scalable machine learning (ML) deployments, including bioinformatics workflows [1]. Lightweight distributions such as K3s improve deployability in resource-constrained edge environments [25]. Prior studies explored GPU scheduling and distributed execution in cloud systems [10, 23, 26], but Kubernetes-based orchestration for distributed genomic basecalling on low-power edge clusters remains largely unexplored.

Existing research has extensively studied basecalling algorithms, deep learning architectures, and energy-efficient AI independently. However, there is still a lack of systematic studies jointly analyzing distributed basecalling performance, energy consumption, and orchestration in edge environments. In particular, the impact of batch-level parallelism and distributed execution on performance–energy trade-offs remains an open research challenge.

3 System Architecture

3.1 Edge Cluster Configuration

For our investigations, we constructed a homogeneous edge computing platform based on NVIDIA Jetson Nano devices, designed for low-power edge applications (Figure 1). Each node featured a quad-core ARM Cortex-A57 CPU, an NVIDIA Maxwell GPU with 128 CUDA cores, and 4 GB of LPDDR4 memory. The platform supports two power modes: a low-power 5 W mode with limited CPU/GPU resources and a 10 W mode enabling full device performance. These modes allow direct evaluation of performance–energy trade-offs in resource-constrained edge environments. One node in the edge cluster served as the Kubernetes control plane, while four worker nodes ran basecalling workloads and monitoring tasks. All nodes were connected via a local area network, ensuring stable, low-latency communication, reflecting typical field deployment scenarios. Since full Kubernetes distributions introduce significant computational and memory overhead, cluster orchestration was implemented using K3s, a lightweight Kubernetes distribution designed for resource-constrained edge environments. Containerization ensured a unified runtime environment.

3.2 Distributed DNA Basecalling

Due to compatibility with ARM-compiled libraries and support for GPU acceleration on Jetson platforms, three basecalling models were selected for experimental evaluation: SACall, RODAN, and Chiron. All represent different deep learning architectural and implementation approaches. Each model was deployed in an isolated container built from a dedicated Docker image that included the required system dependencies and machine learning libraries. The container images were subsequently deployed in the K3s cluster as independent computational units (so-called *pods*), with GPU acceleration enabled via NVIDIA Container Runtime.

For the edge cluster environment, we have developed an algorithm for distributed DNA basecalling (Algorithm 1). The algorithm operates in a master–worker architecture and performs distributed DNA basecalling on a dataset D of size $|D| = \mathit{filesize}$ using N worker nodes. The master node initially partitions the dataset D into N disjoint data subsets D_i, such that $D = \bigcup_{i=1}^{N} D_i$ and $|D_i| = |D|/N$. Each data partition D_i is then transmitted to the corresponding worker node w_i. In this way, we implement node-level task distribution. On each worker w_i, the received data partition D_i is further divided into $n_i = \lceil |D_i|/\mathit{batchsize} \rceil$ batches $B_{i,j}$ of size $\mathit{batchsize}$ (BS). In this way, we introduce batch-level parallelism. Each batch $B_{i,j}$ is independently processed using the basecalling algorithm $\mathcal{B}$, producing partial outputs $O_{i,j}$. The outputs generated on worker w_i are incrementally merged into a local result set $O_i = \bigcup_{j=1}^{n_i} O_{i,j}$. During execution, each worker node collects local performance and resource utilization metrics $\mathcal{M}_i$, including throughput (SPS), execution time T, average GPU and CPU utilization ($U_{\text{GPU}}, U_{\text{CPU}}$), memory consumption RAM, and average power consumption P. After all workers have completed processing, the master node aggregates the local metrics into a global metric set $\mathcal{M} = \text{Aggregate}(\{\mathcal{M}_i\}_{i=1}^{N})$. At the same time, the partial basecalling outputs O_i are aggregated

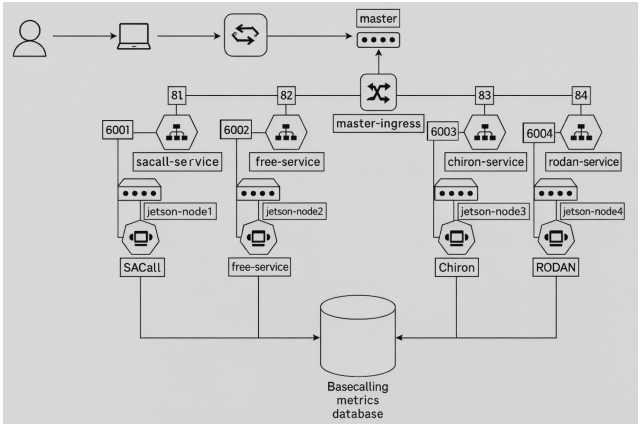


Figure 1: Architecture of the K3s-based Jetson Nano edge computing cluster with DNA basecallers

Algorithm 1 Distributed DNA Basecalling with Batch-level Parallelism and Node-level Task Distribution

Require: number of workers N; dataset D, $|D| = \textit{filesize}$; basecalling algorithm $\mathcal{B}$; batch size BS
Ensure: $\mathcal{M} = \{\text{SPS}, T, U_{\text{GPU}}, U_{\text{CPU}}, RAM, P\}$ metrics
Partition dataset D into N data partitions D_i so that: $D = \bigcup_{i=1}^{N} D_i, \quad |D_i| = |D|/N$
for all $i \in \{1, \ldots, N\}$ **in parallel do**
    Send D_i to worker w_i
end for
for all $i \in \{1, \ldots, N\}$ **in parallel do**
    $n_i \leftarrow \lceil |D_i|/BS \rceil$
    Partition D_i into batches $B_{i,j}$ on worker w_i, such that: $D_i = \bigcup_{j=1}^{n_i} B_{i,j}, \quad |B_{i,j}| = BS$
    for $j \leftarrow 1$ to n_i **do**
        $O_{i,j} \leftarrow \mathcal{B}(B_{i,j})$
        $O_i \leftarrow O_i \cup O_{i,j}$
    end for
    Collect local metrics $\mathcal{M}_i$
end for
Aggregate global metrics and output data:
    $\mathcal{M} \leftarrow \text{Aggregate}\left(\{\mathcal{M}_i\}_{i=1}^{N}\right)$
    $O \leftarrow \text{Aggregate}\left(\{O_i\}_{i=1}^{N}\right)$
Save O
return $\mathcal{M}$

into the final output dataset O, which is subsequently stored. The algorithm returns the aggregated performance metrics $\mathcal{M}$, providing a quantitative evaluation of the distributed basecalling process.

4 Experimental Setup and Evaluation Metrics

The experiments evaluated the impact of selected configuration and system parameters on the performance and energy efficiency of basecalling. We used three independent datasets, representing diverse sequencing use cases and varying input data complexity. The first dataset (0.21 MB per test instance) consisted of RNA signal data in FAST5 format, obtained from a publicly available dataset generated with the Taiyaki framework [17]. The second dataset (~0.64 MB) comprised sequencing reads in the FASTQ format from a clinically relevant *Klebsiella pneumoniae* isolate (strain 033), originally analyzed in the study by Wick et al. [24]. The third dataset (~2.28 MB) was obtained from a publicly available Oxford Nanopore Technologies data repository [18], specifically from the *Q20_ULK_Cliveome* collection, which provides high-accuracy nanopore sequencing reads for benchmarking and large-scale analysis. Five batch size (BS) values corresponding to the number of signal chunks processed simultaneously during inference were analyzed: 16, 32, 64, 128, and 140, with the maximum constrained by device memory limitations. Experiments were conducted in two device power modes, 5 W and 10 W, to investigate performance–energy trade-offs.

To provide a comprehensive evaluation of basecalling performance, a set of metrics was used, including both processing throughput and system resource utilization. The primary performance metric was Samples Per Second (SPS) as a fundamental indicator for efficiency and throughput [14, 24]. SPS enables direct comparison of different models and hardware configurations regardless of dataset size. The SPS value was calculated as follows:

$$\text{SPS} = \frac{|D|}{T}, \tag{1}$$

where: $|D|$ is the total number of signal samples in the dataset, and T is the total processing time. The performance evaluation was complemented by monitoring hardware resource utilization, including GPU, CPU, and RAM. For experiments conducted in the cluster environment, we also performed the horizontal scalability analysis. Two key metrics were determined: scalability factor and parallel efficiency. Parallel efficiency reflects the extent to which the achieved speedup is proportional to the number of computational nodes (formula 2).

$$\text{Efficiency } (E) = \frac{T_1}{N \cdot T_N} \times 100\%, \tag{2}$$

where T_1 is the execution time using a single worker node, and T_N is the execution time using N worker nodes.

The scalability factor enables direct comparison of basecalling execution time between single-node and multi-node cluster configurations (formula 3). In this study, experiments were conducted on an edge cluster comprising $N = 4$ worker nodes.

$$\text{Scalability factor} = \frac{T_1}{T_N}. \tag{3}$$

Energy efficiency represents a critical evaluation criterion for edge computing systems, particularly for mobile deployment scenarios with limited power. In this study, energy efficiency was assessed by relating system power consumption to the number of processed genomic samples, enabling comparison of different models and hardware configurations independently of individual experiment duration. Energy efficiency is expressed as power-normalized throughput:

$$\text{Energy efficiency} = \frac{P \cdot T}{|D|}, \tag{4}$$

where: P is the average power consumption (W), T is the processing time (s), $|D|$ is the total number of processed samples.

5 Results

We performed a series of tests to experimentally evaluate the proposed architecture along several dimensions, including (5.1) the impact of batch size and power mode on the basecalling throughput, (5.2) scalability of the solution for various models, (5.3) resource consumption for batch-level parallelism, and (5.4) energy efficiency

5.1 Impact of Batch Size and Power Mode on Throughput

The impact of batch size on basecalling throughput was first evaluated under the high-performance configuration (10 W, mode 0) to isolate workload-scaling effects from power-related constraints (Table 1). Here, SACall demonstrated clear sensitivity to batch-level parallelism: for the 2.28 MB dataset, throughput peaked at 35,353 SPS at BS=64 (a 34% improvement over BS=16), then declined to 28,318 SPS at BS=140 due to resource saturation, with optimal

Table 1: Throughput obtained in the high-performance mode (SPS_0) and relative decrease (Δ%) when working in the low-power mode for various basecalling models, datasets, and batch sizes

		BS=16		BS=32		BS=64		BS=128		BS=140	
Model	**Data**	SPS_0	Δ%	SPS_0	Δ%	SPS_0	Δ%	SPS_0	Δ%	SPS_0	Δ%
Chiron	0.21	235	-46.4	416	-44.7	2 937	-19.6	4 064	-30.7	4 567	-23.5
	0.64	706	-46.1	1 267	-46.3	4 735	-9.4	5 421	-11.3	7 651	-25.2
	2.28	3 695	-16.0	4 577	-21.5	11 105	-49.8	16 015	-46.6	16 054	-45.0
Rodan	0.21	5 325	-33.0	5 876	-38.7	6 575	-36.8	6 235	-31.9	5 937	-26.0
	0.64	15 265	-26.1	8 738	-10.2	6 120	-10.6	5 832	-18.2	5 416	-15.2
	2.28	10 815	-4.3	10 622	-3.4	10 120	-17.2	9 875	-18.6	9 227	-14.8
SACall	0.21	4 010	-25.3	3 602	-10.0	8 738	-55.1	10 389	-63.8	9 684	-60.1
	0.64	9 245	-1.7	9 546	-4.1	10 378	-7.3	10 590	-12.7	10 427	-4.9
	2.28	26 295	-25.1	27 825	-22.2	35 353	-31.8	33 033	-28.8	28 318	-12.5

performance consistently falling within the BS=64–128 range across all datasets. Chiron exhibited the strongest positive scaling trend, with throughput increasing monotonically from 3,695 SPS (BS=16) to 16,054 SPS (BS=140) on the 2.28 MB dataset — a 4.35-fold improvement — indicating that the model effectively amortizes per-batch overhead without reaching saturation within the evaluated range. Conversely, RODAN displayed limited or negative scaling behavior: for both the 2.28 MB and 0.64 MB datasets, peak throughput was observed at BS=16 and declined progressively with increasing batch size, with only a moderate peak at BS=64 recorded for the smallest dataset.

To further assess the role of power constraints, identical experiments were repeated under the low-power mode (5 W, mode 1). The 10 W configuration consistently yielded higher throughput across all models and datasets. However, the magnitude of degradation for the 5 W mode proved to be strongly model-dependent. Chiron exhibited the greatest sensitivity, with throughput decreasing from 16,054 SPS in 10 W mode to below 9,000 SPS in 5 W mode for the largest dataset — a reduction of ∼ 45% — with similar relative declines observed for smaller datasets, indicating pronounced dependence on GPU frequency and CPU resources. SACall demonstrated moderate sensitivity (up to 31.8% reduction for the 2.28 MB dataset) while retaining the highest absolute throughput among all models in both power configurations. RODAN proved to be the least affected, with throughput decreasing from 10,815 SPS to ∼ 10, 350 SPS at BS=16 for the largest dataset (a 4.3% reduction), suggesting that its performance is primarily governed by algorithmic characteristics rather than available computational power.

In summary, both batch size and power mode constitute critical, model-dependent factors governing basecalling throughput in edge environments. Chiron achieves the strongest batch-size scaling yet incurs the largest performance penalty under power constraints. SACall reaches optimal throughput at moderate batch sizes and sustains competitive performance under reduced power budgets. RODAN, while exhibiting limited scalability, maintains the most stable behavior across power configurations. These findings underscore the necessity of joint batch-size and power-mode tuning when deploying basecalling workloads on resource-constrained edge platforms.

5.2 Single Node vs Distributed Computing

To evaluate the horizontal scalability of the proposed edge cluster, execution time was analyzed as a function of the number of active compute nodes (one to four), and the parallel speedup (S) and efficiency (E) were calculated as defined in Section 4. A fixed BS=128 was used in all scalability experiments, as this configuration provided the most stable throughput across all evaluated models (Table 1, the second highest SPS for almost all cases).

As shown in Table 2, all models benefit from additional compute nodes, but scaling characteristics differ significantly. The highest scalability was observed for Chiron, which achieved a speedup of 4.97 on 4 nodes (execution time reduced from 174 s to 35 s) with a parallel efficiency of 124.3%, indicating superlinear speedup. This behavior may result from improved memory locality and more efficient GPU utilization due to smaller data partitions per node. SACall achieved a speedup of 2.63 on four nodes (62% time reduction), with parallel efficiency decreasing from 81.9% (two nodes) to 65.8% (four nodes), reflecting growing communication and synchronization overhead. RODAN demonstrated the lowest scalability, with a speedup of only 1.86 and efficiency of 46.5% on four nodes, likely due to sequential dependencies within its recurrent components. Overall, the results demonstrate that horizontal scaling of basecalling workloads in edge computing environments is strongly dependent on the underlying model architecture. Computationally intensive models such as Chiron can benefit substantially from workload distribution across multiple nodes, while models with more limited internal parallelism, such as RODAN, achieve comparatively smaller performance gains.

5.3 Resource Utilization Analysis

In addition to throughput, the experiments also analyzed the utilization of computational resources during the basecalling process.

GPU utilization (Figure 2) varied substantially across models. Chiron demonstrated the most consistent increase, with average GPU load rising from ∼ 26.7% at BS=16 to over 62% at BS=140. This near-monotonic trend correlates with the substantial throughput improvements reported in Section 5.1, suggesting that the model benefits from improved GPU occupancy at larger workloads. SACall exhibited a more irregular profile, with GPU usage ranging between 9% and 20% and peaking at BS=64 (20.1%). The relatively narrow

Table 2: Execution time, scalability factor (S), and parallel efficiency (E)

Nodes	SACall			Rodan			Chiron		
	Time (s)	S	E (%)	Time (s)	S	E (%)	Time (s)	S	E (%)
1	50.0	1.00	100	47.0	1.00	100	174.0	1.00	100
2	30.5	1.64	81.9	32.5	1.45	72.3	83.0	2.10	104.8
3	23.3	2.15	71.6	29.3	1.60	53.3	52.3	3.33	111.0
4	19.0	2.63	65.8	25.25	1.86	46.5	35.0	4.97	124.3

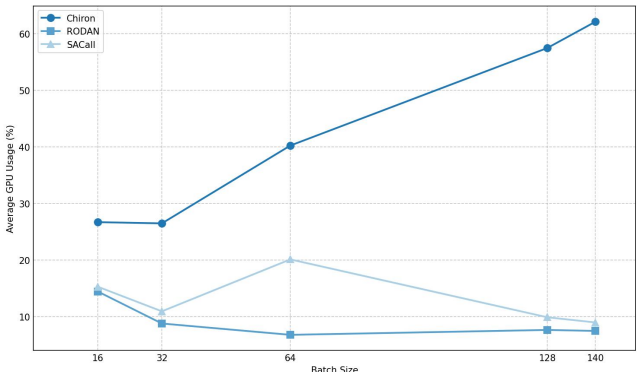


Figure 2: Average GPU utilization as a function of batch size for the evaluated basecalling models.

variation range (∼ 11%) suggests that GPU utilization in SACall is influenced by additional architectural factors. RODAN presented the lowest overall GPU utilization (below 15%), which gradually decreased with increasing batch size, aligning with the limited throughput scaling and suggesting that the architecture does not fully exploit the available GPU parallelism on the evaluated edge platform.

Table 3: Average CPU and RAM utilization depending on batch size for the evaluated basecalling models

BS	CPU (%)			RAM (GB)		
	SACall	RODAN	Chiron	SACall	RODAN	Chiron
16	30.77	68.24	72.56	2.88	2.77	2.28
32	28.85	55.96	68.00	2.87	2.72	2.29
64	32.14	46.71	57.86	2.89	3.21	2.42
128	29.50	58.90	55.19	2.90	3.48	2.44
140	28.12	62.86	50.67	2.93	3.57	2.47

CPU utilization (Table 3, left) revealed complementary patterns. For Chiron, CPU usage decreased consistently from 72.56% (BS=16) to 50.67% (BS=140), inversely mirroring the GPU utilization increase and indicating effective workload offloading to the GPU at larger batch sizes. RODAN exhibited the highest CPU usage at BS=16 (68.24%), corresponding to its peak throughput. CPU utilization decreased to 46.7% at BS=64 before rising again, suggesting that the architecture relies more heavily on CPU-side processing and does not proportionally benefit from larger batches. In contrast, SACall maintained highly stable CPU utilization close to 30% regardless of batch size. This stable utilization profile suggests that the model distributes computational tasks between CPU and GPU in a balanced manner.

Memory utilization (Table 3, right) further highlights architectural differences. RODAN showed the highest sensitivity, with RAM increasing from 2.77 GB to 3.57 GB (+29%), which may contribute to reduced throughput at larger batch sizes on resource-constrained devices. Chiron exhibited modest growth from 2.28 GB to 2.47 GB (+8%), suggesting efficient memory management even at larger input batches. SACall demonstrated the most stable footprint, varying only from 2.87 GB to 2.93 GB (+1.7%).

Taken together, the combined analysis of GPU, CPU, and memory utilization provides additional insight into the performance characteristics of the evaluated basecalling models. While Chiron benefits from increased GPU utilization and efficient memory management at larger batch sizes, RODAN exhibits higher CPU dependency and increased memory consumption. SACall, on the other hand, maintains relatively stable resource utilization across all configurations, which explains its moderate but consistent throughput scaling observed in Section 5.1.

5.4 Energy Efficiency Evaluation

Energy consumption is a critical factor when deploying edge computing infrastructures, particularly in environments where access to electrical power is limited. To evaluate the energy characteristics of the proposed system, power consumption and energy efficiency (calculated according to Eq. 4) were measured for all models under the high-performance configuration across all three datasets. The average power consumption (Figure 3) correlates with computational resource utilization discussed in Section 5.3. Chiron exhibited the highest power consumption, increasing from approximately 6.99 W at BS=16 to 8.21 W at BS=140, consistent with its growing GPU utilization at input batches. In contrast, RODAN demonstrated relatively stable consumption close to 5 W across most configurations, with a moderate increase to 5.67 W at BS=128. SACall maintained the lowest and most stable power usage (4.0–4.3 W). This behavior aligns with the previously observed stability in CPU and memory utilization as well as the moderate GPU workload.

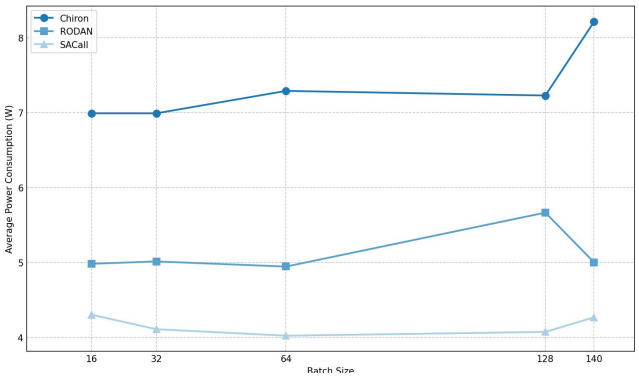


Figure 3: Power consumption depending on the batch size.

While power consumption reflects instantaneous system load, energy efficiency — expressed as the energy required to process a single sample (Table 4) — provides a more comprehensive metric. SACall demonstrates the most stable energy efficiency profile, with values remaining close to 1.6 J across all batch sizes. RODAN achieves the lowest energy cost per sample at BS=16 (0.65 J). However, as batch size increases, the cost grows to 1.35 J at BS=64 before stabilizing at approximately 1.10 J, reflecting the throughput decline that is not accompanied by a proportional reduction in power consumption. The most pronounced improvement is observed for Chiron. The model initially exhibits the highest energy cost per sample (11.0 J at BS=16), but the value decreases significantly as the batch size grows, reaching 3.41 J at BS=140. This improvement corresponds directly to the strong throughput scaling reported in Section 5.1 and the increasing GPU utilization described in Section 5.3, indicating that larger batches enable more efficient utilization of the available hardware resources.

Table 4: Energy efficiency of basecalling models

BS	**SACall**	**RODAN**	**Chiron**
16	1.60 J	0.65 J	11.00 J
32	1.58 J	0.92 J	6.00 J
64	1.55 J	1.35 J	4.33 J
128	1.62 J	1.10 J	3.50 J
140	1.65 J	1.10 J	3.41 J

Overall, energy efficiency in edge-based basecalling is strongly influenced by the interaction between model architecture, batch size, and hardware utilization. Models capable of exploiting GPU parallelism, such as Chiron, significantly improve efficiency at larger workloads, while architectures with limited scalability, such as RODAN, show smaller gains despite similar power levels. These findings highlight the importance of jointly considering throughput, resource utilization, and energy efficiency when selecting basecalling models for deployment on resource-constrained edge clusters.

6 Discussion and Conclusions

This study presents a system-level evaluation of distributed nanopore basecalling on low-power edge platforms, jointly analyzing scalability, resource utilization, and energy efficiency. In contrast to prior work focused mainly on accuracy or isolated performance metrics [20, 24], the proposed framework evaluates the interaction between workload configuration, distributed execution, and energy consumption under realistic edge deployment conditions. The results demonstrate that basecalling behavior strongly depends on model architecture and workload configuration. SACall provides the most balanced trade-off between throughput, scalability, and energy efficiency, making it well suited for resource-constrained edge deployments. Chiron achieves the highest throughput scaling but at significantly higher GPU utilization and energy cost, whereas RODAN performs efficiently only for smaller workloads. Similar to observations reported for distributed machine learning systems [10], the experiments also reveal diminishing scalability gains due to orchestration and parallelization overhead. The presented findings are particularly relevant for mobile and field genomic applications, including outbreak monitoring and remote health deployments [5], where energy availability and infrastructure constraints directly affect operational feasibility. Overall, the study shows that efficient edge-based genomic inference requires balancing model architecture, batch-level parallelism, and system-level resource constraints rather than optimizing throughput alone.

References

[1] Augustyn, D. R., Wyciślik, , and Mrozek, D. Perspectives of using cloud computing in integrative analysis of multi-omics data. *Briefings in Functional Genomics 20*, 4 (07 2021), 198–206.

[2] Boers, S., et al. Understanding and overcoming the pitfalls and biases of next-generation sequencing. *European Journal of Clinical Microbiology* (2019).

[3] Boykin, L. M., et al. Tree Lab: Portable genomics for early detection of plant viruses and pests in sub-Saharan Africa. *Genes 10*, 9 (2019), 632.

[4] Burns, B., et al. Borg, Omega, and Kubernetes. *Commun. ACM 59*, 5 (2016), 50–57.

[5] Castro-Wallace, et al. Nanopore DNA sequencing and genome assembly on the International Space Station. *Scientific Reports 7* (2017), 18022.

[6] Cheng, Y., Wang, D., Zhou, P., and Zhang, T. Model compression and acceleration for deep neural networks: The principles, progress, and challenges. *IEEE Signal Processing Magazine 35*, 1 (2018), 126–136.

[7] Daghero, F., Pagliari, D. J., and Poncino, M. Energy-efficient deep learning inference on edge devices. In *Advances in Computers*, vol. 117. Elsevier, 2020, pp. 247–301.

[8] David, M., et al. Nanocall: an open source basecaller for Oxford Nanopore sequencing data. *Bioinformatics 33*, 1 (2017), 49–55.

[9] Grzesik, P., and Mrozek, D. Metagenomic analysis at the edge with Jetson Xavier NX. In *Computational Science – ICCS 2021* (2021), vol. 12745 of *Lecture Notes in Computer Science*, Springer, pp. 500–511.

[10] Hong, C.-H., Spence, I., and Nikolopoulos, D. S. GPU virtualization and scheduling methods: A comprehensive survey. *ACM Comput. Surv. 50*, 3 (2017).

[11] Huang, N., Nie, F., Ni, P., Luo, F., and Wang, J. SACall: A neural network basecaller for Oxford Nanopore Sequencing data based on self-attention mechanism. *IEEE/ACM Trans. on Comp. Biology and Bioinformatics 19*, 1 (2022), 614–623.

[12] Li, E., Zeng, L., Zhou, Z., and Chen, X. Edge AI: On-demand accelerating deep neural network inference via edge computing. *IEEE Transactions on Wireless Communications 19*, 1 (2020), 447–457.

[13] Morabito, R., Cozzolino, V., Ding, A. Y., Beijar, N., and Ott, J. Consolidate IoT edge computing with lightweight virtualization. *IEEE Network 32*, 1 (2018), 102–111.

[14] Neumann, D., Reddy, A., and Ben-Hur, A. RODAN: a fully convolutional architecture for basecalling nanopore RNA sequencing data. *BMC Bioinformatics 23*, 142 (2022).

[15] Oxford Nanopore Technologies . Dorado basecaller, 2023. https://nanoporetech.com.

[16] Oxford Nanopore Technologies,. Bonito: A PyTorch basecaller for Oxford Nanopore sequencing. https://github.com/nanoporetech/bonito, 2019.

[17] Oxford Nanopore Technologies. Taiyaki: training framework and example datasets for nanopore basecalling, 2020.

[18] Oxford Nanopore Technologies. Oxford nanopore open data: Q20 ulk cliveome dataset. https://registry.opendata.aws/ont-open-data/, 2021. Accessed: 2026-04-30.

[19] Oxford Nanopore Technologies. Guppy basecaller, 2022. https://nanoporetech.com.

[20] Perešíni, P., Boža, V., Brejová, B., and Vinař, T. Nanopore base calling on the edge. *Bioinformatics 37*, 24 (2021), 4661–4667.

[21] Sze, V., Chen, Y.-H., Yang, T.-J., and Emer, J. S. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE 105*, 12 (2017), 2295–2329.

[22] Teng, H., et al. Chiron: translating nanopore raw signal directly into nucleotide sequence using deep learning. *GigaScience 7*, 5 (2018), giy037.

[23] Ukidave, Y., Li, X., and Kaeli, D. Mystic: Predictive scheduling for GPU Based cloud servers using machine learning. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2016), pp. 353–362.

[24] Wick, R. R., et al. Performance of neural network basecalling tools for Oxford Nanopore sequencing. *Genome Biology 20*, 129 (2019).

[25] Yakubov, D., and Hästbacka, D. Comparative analysis of lightweight Kubernetes distributions for edge computing: Performance and resource efficiency. In *Service-Oriented and Cloud Computing: 11th IFIP WG 6.12 European Conference, ESOCC 2025, Bolzano, Italy, February 20–21, 2025, Proceedings* (Berlin, Heidelberg, 2025), Springer-Verlag, p. 81–95.

[26] Zhao, Y., Lei, S., Zhang, G., and Geng, Y. Design and development of container-based cloud platform for power equipment simulation. *Comp. Engin. 47*, 9 (2021).